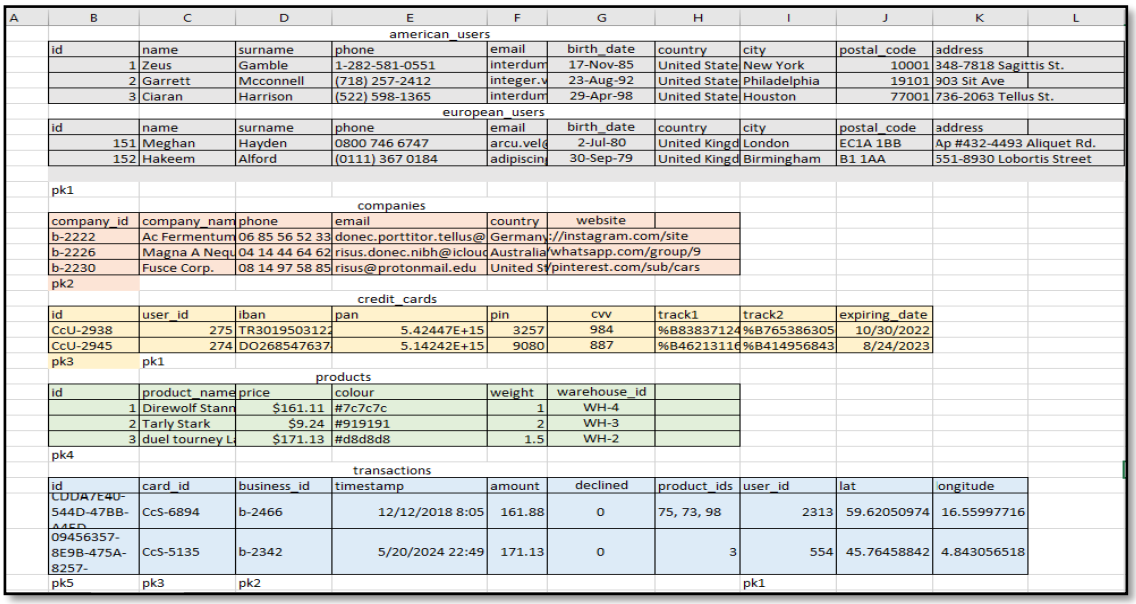
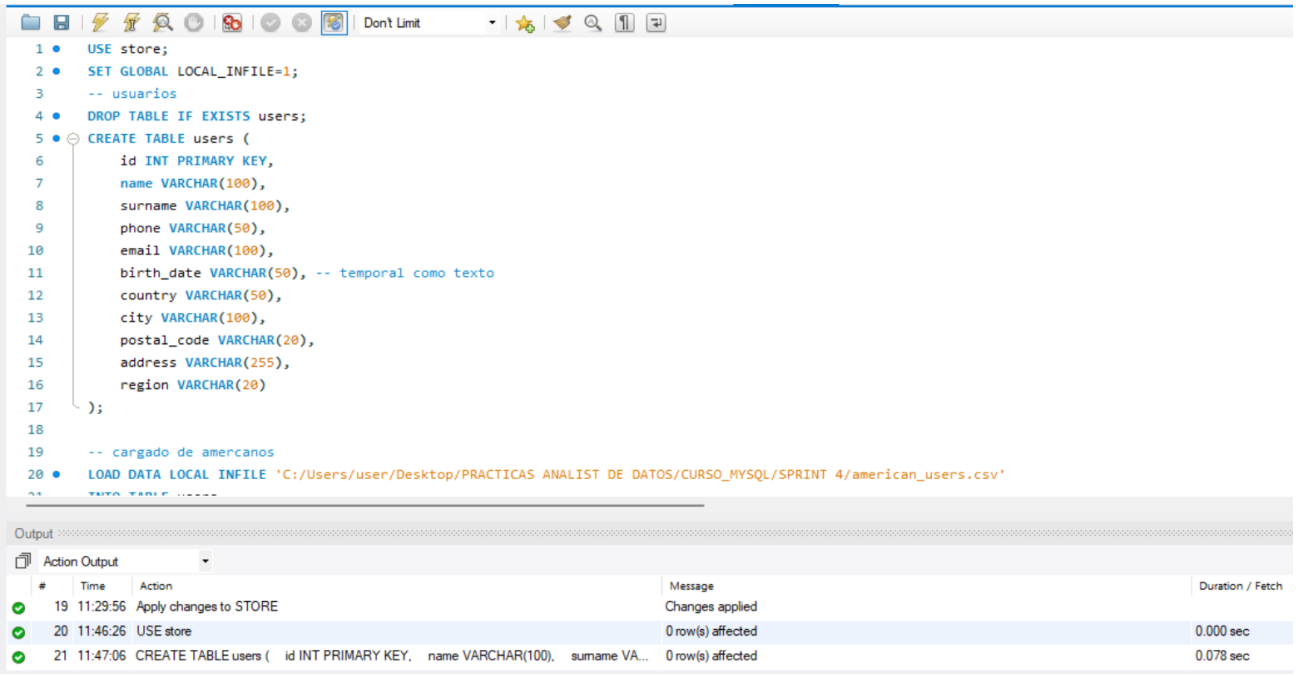
Previo a resolver los ejercicios, realicé un análisis de los datos en Excel, donde pude apreciar las tablas y determinar las PK y FK correspondientes como se aprecia en la siguiente imagen.



A	B	C	D	E	F	G	H	I	J	K	L
				american_users							
	id	name	surname	phone	email	birth_date	country	city	postal_code	address	
	1	Zeus	Gamble	1-282-581-0551	interdum	17-Nov-85	United State	New York	10001	348-7818 Sagittis St.	
	2	Garrett	Mcconnell	(718) 257-2412	integer.v	23-Aug-92	United State	Philadelphia	19101	903 Sit Ave	
	3	Ciaran	Harrison	(522) 598-1365	interdum	29-Apr-98	United State	Houston	77001	736-2063 Tellus St.	
				european_users							
	id	name	surname	phone	email	birth_date	country	city	postal_code	address	
	151	Meghan	Hayden	0800 746 6747	arcu.vel@	2-Jul-80	United Kingd	London	EC1A 1BB	Ap #432-4493 Aliquet Rd.	
	152	Hakeem	Alford	(0111) 367 0184	adipiscin	30-Sep-79	United Kingd	Birmingham	B1 1AA	551-8930 Lobortis Street	
	pk1										
				companies							
	company_id	company_nam	phone	email	country	website					
	b-2222	Ac Fermentum	06 85 56 52 33	donec.porttitor.tellus@	Germany	://instagram.com/site					
	b-2226	Magna A Nequ	04 14 44 64 62	risus.donec.nibh@icloud	Australia	whatsapp.com/group/9					
	b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United St	pinterest.com/sub/cars					
	pk2										
				credit_cards							
	id	user_id	iban	pan	pin	cvv	track1	track2	expiring_date		
	CcU-2938	275	TR301950312	5.42447E+15	3257	984	%B83837124	%B765386305	10/30/2022		
	CcU-2945	274	DO268547637	5.14242E+15	9080	887	%B46213116	%B414956843	8/24/2023		
	pk3	pk1									
				products							
	id	product_name	price	colour	weight	warehouse_id					
	1	Direwolf Stann	$161.11	#7c7c7c	1	WH-4					
	2	Tarly Stark	$9.24	#919191	2	WH-3					
	3	duel tourney L	$171.13	#d8d8d8	1.5	WH-2					
	pk4										
				transactions							
	id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	ongitude	
	CDDA7E40-544D-47BB-A45D-	CcS-6894	b-2466	12/12/2018 8:05	161.88	0	75, 73, 98	2313	59.62050974	16.55997716	
	09456357-8E9B-475A-8257-	CcS-5135	b-2342	5/20/2024 22:49	171.13	0	3	554	45.76458842	4.843056518	
	pk5	pk3	pk2					pk1			

Así mismo use DROP TABLE como instrucción alternativa por si requiero comenzar nuevamente y eliminar la tabla completa. Posteriormente se realiza la estructura de la tabla users, así cargarlo con los datos de los CSV que corresponden, no obstante, american_users y european_users cuentan con los mismos campos, por eso motivo se unió ambas tablas, agregando un campo adicional de región, para distinción cuando se unan.



```sql
USE store;
SET GLOBAL LOCAL_INFILE=1;
-- usuarios
DROP TABLE IF EXISTS users;
CREATE TABLE users (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(50),
    email VARCHAR(100),
    birth_date VARCHAR(50), -- temporal como texto
    country VARCHAR(50),
    city VARCHAR(100),
    postal_code VARCHAR(20),
    address VARCHAR(255),
    region VARCHAR(20)
);

-- cargado de amercanos
LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/american_users.csv'
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
19	11:29:56	Apply changes to STORE	Changes applied	
20	11:46:26	USE store	0 row(s) affected	0.000 sec
21	11:47:06	CREATE TABLE users (id INT PRIMARY KEY, name VARCHAR(100), surname VA...	0 row(s) affected	0.078 sec

```sql
-- cargado de amercanos
LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/american_users.csv'
INTO TABLE users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(
id, name, surname, phone, email,
birth_date, country, city, postal_code, address
)
SET region = 'America';

-- cargada de europeos
LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/european_users.csv'
INTO TABLE users
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(
id, name, surname, phone, email,
birth_date, country, city, postal_code, address
)
SET region = 'Europe';
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
22	11:48:58	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.156 sec
23	11:49:42	SELECT * FROM americans_users	Error Code: 1146. Table 'store.americans_users' doesn't exist	0.000 sec
24	11:51:33	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.610 sec

Una vez realizado la estructura y la carga de datos, prosigo a la verificación de la tabla:

```sql
SELECT * FROM users;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	name	surname	phone	email	birth_date	country	city	postal_code	address	region
1	Zeus	Gamble	1-282-581-0551	interdum.enim@protonmail.edu	Nov 17, 1985	United States	New York	10001	348-7818 Sagittis St.	America
2	Garrett	Mcconnell	(718) 257-2412	integer.vitae.nibh@protonmail.org	Aug 23, 1992	United States	Philadelphia	19101	903 Sit Ave	America
3	Ciaran	Harrison	(522) 598-1365	interdum.feugiat@aol.org	Apr 29, 1998	United States	Houston	77001	736-2063 Tellus St.	America
4	Howard	Stafford	1-411-740-3269	ornare.egestas@icloud.edu	Feb 18, 1989	United States	Phoenix	85001	Ap #545-2244 Erat. Rd.	America
5	Hayfa	Pierce	1-554-541-2077	et.malesuada.fames@hotmail.org	Sep 26, 1998	United States	Philadelphia	19101	341-2821 Ultrices Av.	America
6	Joel	Tyson	(718) 288-8020	gravida.nunc.sed@yahoo.ca	Oct 15, 1989	United States	San Jose	95101	888-2799 Amet Street	America
7	Rafael	Jimenez	(817) 689-0478	eget@outlook.ca	Dec 4, 1981	United States	Chicago	60601	8627 Malesuada Rd.	America
8	Nissim	Franks	(692) 157-3469	egestas.aliquam.fringilla@google.ca	Aug 1, 1993	United States	New York	10001	Ap #251-7144 Integer St.	America
9	Mannix	Mcclain	(590) 883-2184	aliquam.nisl@outlook.com	Jan 24, 1987	United States	San Antonio	78201	647-3080 Lacus. St.	America
10	Robert	Mccarthy	(324) 746-6771	fermentum@protonmail.com	Apr 30, 1984	United States	San Jose	95101	P.O. Box 773	America
11	Joan	Baird	(981) 429-8106	et@outlook.net	Feb 25, 1990	United States	Los Angeles	90001	P.O. Box 687	America

users 1

Output

Action Output

#	Time	Action	Message
23	11:49:42	SELECT * FROM americans_users	Error Code: 1146. Table 'store.americans_users' doesn't exist
24	11:51:33	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0
25	11:55:00	SELECT * FROM users	5000 row(s) returned

Al observar los datos, continue con la limpieza o modificación de datos para mejorar el procesamiento, en particular las fechas que se normalizo el formato.

```sql
-- limpieza
UPDATE users
SET birth_date = CASE
    WHEN birth_date LIKE '%,%'
        THEN STR_TO_DATE(birth_date, '%b %d, %Y')
    WHEN birth_date LIKE '%-%'
        THEN STR_TO_DATE(birth_date, '%d-%b-%y')
    ELSE NULL
END;
```

Output

Action Output

#	Time	Action	Message
24	11:51:33	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0
25	11:55:00	SELECT * FROM users	5000 row(s) returned
26	11:57:09	UPDATE users SET birth_date = CASE WHEN birth_date LIKE '%,%' THEN STR_T...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0

Repetimos el proceso con las demás tablas, tomando en cuenta que los campos son diferentes y prestando atención a los datos.

Tabla companies

```sql
-- companies
DROP TABLE IF EXISTS companies;
CREATE TABLE companies (
    company_id VARCHAR(20) PRIMARY KEY,
    company_name VARCHAR(100),
    phone VARCHAR(50),
    email VARCHAR(100),
    country VARCHAR(50),
    website VARCHAR(100)
);
LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/companies.csv'
INTO TABLE companies
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Output

Action Output

#	Time	Action	Message
30	11:59:27	DROP TABLE IF EXISTS companies	0 row(s) affected
31	11:59:31	CREATE TABLE companies (company_id VARCHAR(20) PRIMARY KEY, company_n...	0 row(s) affected
32	11:59:35	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0

Tabla credit_cards

```sql
-- credit_cards tabla estructura
DROP TABLE IF EXISTS credit_cards;
CREATE TABLE credit_cards (
    id VARCHAR(20) PRIMARY KEY,
    user_id INT,
    iban VARCHAR(50),
    pan VARCHAR(50),
    pin VARCHAR(10),
    cvv VARCHAR(10),
    track1 TEXT,
    track2 TEXT,
    expiring_date VARCHAR(20),
    FOREIGN KEY (user_id) REFERENCES users(id)
);
-- carga de datos a credit cards
LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/credit_cards.csv'
INTO TABLE credit_cards
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Output

Action Output

#	Time	Action	Message
33	12:02:16	DROP TABLE IF EXISTS credit_cards	0 row(s) affected, 1 warning(s): 1051 Unknown table 'store.credit_cards'
34	12:02:26	CREATE TABLE credit_cards (id VARCHAR(20) PRIMARY KEY, user_id INT, iban ...	0 row(s) affected
35	12:02:31	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0

Continuando nuevamente procedo a limpiar y normalizar la fecha.

```sql
-- limpieza de credit_cards
UPDATE credit_cards
SET expiring_date = CASE
    WHEN expiring_date LIKE '%/%/%'
        THEN STR_TO_DATE(expiring_date, '%m/%d/%y')
    WHEN expiring_date LIKE '%-%'
        THEN STR_TO_DATE(expiring_date, '%m-%y')
    ELSE NULL
END;
```

Output

Action Output

#	Time	Action	Message
34	12:02:26	CREATE TABLE credit_cards (id VARCHAR(20) PRIMARY KEY, user_id INT, iban ...	0 row(s) affected
35	12:02:31	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0
36	12:04:23	UPDATE credit_cards SET expiring_date = CASE WHEN expiring_date LIKE '%/%/%' ...	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0

Table products

```sql
-- produts
DROP TABLE IF EXISTS products;
CREATE TABLE products (
    id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price VARCHAR(20),
    colour VARCHAR(20),
    weight VARCHAR(20),
    warehouse_id VARCHAR(20)
);

LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/products.csv'
INTO TABLE products
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
-- limpieza
UPDATE products
SET
    price = REPLACE(price, '$', ''),
    weight = REPLACE(weight, 'kg', '');
```

Output

Action Output

#	Time	Action	Message
38	12:06:47	CREATE TABLE products (id INT PRIMARY KEY, product_name VARCHAR(100), p...	0 row(s) affected
39	12:06:56	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0
40	12:07:08	UPDATE products SET price = REPLACE(price, '$', ''), weight = REPLACE(weight, 'kg', '')	100 row(s) affected Rows matched: 100 Changed: 100 Warnings: 0

Tabla transactions

```sql
CREATE TABLE transactions (
    id VARCHAR(50) PRIMARY KEY,
    card_id VARCHAR(20),
    business_id VARCHAR(20),
    timestamp VARCHAR(50),
    amount VARCHAR(50),
    declined VARCHAR(10),
    product_ids VARCHAR(255),
    user_id INT,
    lat VARCHAR(50),
    longitude VARCHAR(50)
);
-- carga de datos transactions
LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/CURSO_MYSQL/SPRINT 4/transactions.csv'
INTO TABLE transactions
FIELDS TERMINATED BY ';'
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Output

Action Output

#	Time	Action	Message
63	12:38:13	CREATE TABLE transactions (id VARCHAR(50) PRIMARY KEY, card_id VARCHAR(2...	0 row(s) affected
64	12:38:20	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0
65	12:38:56	UPDATE transactions SET timestamp = CASE WHEN timestamp LIKE '%-%-% %:%:%...	0 row(s) affected Rows matched: 100000 Changed: 0 Warnings: 0

```sql
-- limpieza de trtansactions
UPDATE transactions
SET
    timestamp = CASE
        WHEN timestamp LIKE '%-%-% %:%:%'
            THEN STR_TO_DATE(timestamp, '%Y-%m-%d %H:%i:%s')
        WHEN timestamp LIKE '%/%/% %:%'
            THEN STR_TO_DATE(timestamp, '%d/%m/%Y %H:%i')
        ELSE NULL
    END,
    amount = REPLACE(amount, '$', ''),
    declined = CASE
        WHEN declined IN ('1','true','TRUE') THEN 1
        ELSE 0
    END;
```

Output

Action Output

#	Time	Action	Message
63	12:38:13	CREATE TABLE transactions (id VARCHAR(50) PRIMARY KEY, card_id VARCHAR(2...	0 row(s) affected
64	12:38:20	LOAD DATA LOCAL INFILE 'C:/Users/user/Desktop/PRACTICAS ANALIST DE DATOS/C...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0
65	12:38:56	UPDATE transactions SET timestamp = CASE WHEN timestamp LIKE '%-%-% %:%:%...	0 row(s) affected Rows matched: 100000 Changed: 0 Warnings: 0

```sql
SELECT * FROM transactions;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude
00043A49-2949-494B-A5DD-A5BAE3BB19DD	CcS-9294	b-2458	2024-08-28 07:16:46	395.43	0	16, 26, 97, 87	4713	46.19992926158272	1.4355402821327607
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	155.63	0	66, 69, 87	438	41.59720554463741	12.22175994259365
00045D6B-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2020-07-14 15:37:45	326.01	0	30, 11, 16, 81	2118	29.757295899964348	-95.37963676492454
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2017-09-04 19:44:53	161.6	0	72	2115	53.54888376797025	-113.50305274646564
00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	2017-01-05 18:19:25	148.91	0	18	3025	52.20836951654172	5.690806474241335
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CcU-3358	b-2598	2023-09-23 04:51:43	294.59	0	35, 33, 19	215	53.553485560256014	-113.4991339874781
0009A151-9BCF-4E31-9053-A468FF77FAAB	CcS-7509	b-2546	2023-12-31 00:06:36	383.63	0	93, 55, 28, 91	2928	51.93615735576977	5.342650184655181
0009D494-6245-4DF9-955D-2C084191CFFB	CcS-8483	b-2526	2017-07-18 07:52:02	197.8	0	55, 8, 72	3902	45.49200875032752	-73.57063686984957
000A1DEC-CDB6-4AB2-A619-71DAB8D4A262	CcS-6467	b-2558	2018-09-08 05:29:58	339.94	0	46, 56, 73	1886	55.74247746345254	-3.300093680889247
000A1E64-1414-40B0-9D92-5678A4D958E2	CcS-5966	b-2550	2022-09-17 04:02:19	369.71	0	6, 89, 19	1385	52.08214987185822	5.284238674948618
000A5879-3472-41D9-AF60-42D3503B543C	CcU-4569	b-2590	2020-02-07 23:03:45	162.43	0	58, 61, 55	42	39.94902273793568	-75.17193552521194

transactions 2

Output

Action Output

#	Time	Action	Message
65	12:38:56	UPDATE transactions SET timestamp = CASE WHEN timestamp LIKE '%-%-% %:%:%...	0 row(s) affected Rows matched: 100000 Changed: 0 Warnings: 0
66	12:41:19	SELECT * FROM ATRANSACTIONS	Error Code: 1146. Table 'store.atransactions' doesn't exist
67	12:41:35	SELECT * FROM transactions	100000 row(s) returned

NIVEL 1. Ejercicio 1: Realice una subconsulta que muestre a todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas, para realizar la consulta se veo adecuado resolverlo por joins

```sql
SELECT
    u.id,
    u.name,
    u.surname,
    COUNT(t.id) AS total_transactions
FROM users u
JOIN transactions t ON u.id = t.user_id
GROUP BY u.id, u.name, u.surname
HAVING COUNT(t.id) > 80;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	name	surname	total_transactions
318	Bnyr	Astuw	91
289	Dxwgi	Hwcru	94
454	Sfzzoh	Xgvfridxs	81
185	Molly	Gilliam	110

Result 5

Output

Action Output

#	Time	Action	Message
68	12:46:11	SELECT * FROM users u WHERE u.id IN (SELECT t.user_id FROM transactions t GROUP ...	4 row(s) returned
69	12:48:32	SELECT u.id, u.name, u.surname, COUNT(t.id) AS total_transactions FROM users...	4 row(s) returned
70	12:49:35	SELECT u.id, u.name, u.surname, COUNT(t.id) AS total_transactions FROM users...	4 row(s) returned

Ejercicio 2: Muestra el monto promedio del propietario de una tarjeta de crédito IBAN en Donec Ltd, utiliza al menos 2 tablas.

```sql
SELECT
    c.company_name,
    cc.iban,
    AVG(t.amount) AS avg_amount
FROM transactions t
JOIN credit_cards cc ON t.card_id = cc.id
JOIN companies c ON t.business_id = c.company_id
WHERE c.company_name = 'Donec Ltd'
GROUP BY  cc.iban;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	iban	avg_amount
Donec Ltd	XX9114064011255863075868 05	356.24666666666667
Donec Ltd	SK9446370242474562577506	142.96
Donec Ltd	XX7767529178459529755556 40	257.37
Donec Ltd	XX4138273622897193049089 90	139.59
Donec Ltd	XX3477872460707696107803 08	240.41
Donec Ltd	XX6887684365430908948546 02	188.58
Donec Ltd	MK28368851538688349	439.39
Donec Ltd	PL76249283566852676343404576	541.56

Result 6

Output

Action Output

#	Time	Action	Message
70	12:49:35	SELECT u.id, u.name, u.surname, COUNT(t.id) AS total_transactions FROM users...	4 row(s) returned

Nivel 2 ejercicio 1: Cree una nueva tabla que refleje el estado de las tarjetas de crédito según Si las últimas tres transacciones han sido rechazadas, entonces está inactivo; si al menos una no es rechazada, entonces está activa. Comenzando en esta tabla, responde: ¿Cuántas tarjetas están activas?

```sql
CREATE TABLE card_status AS
WITH ranked AS (
    SELECT
        t.card_id,
        t.declined,
        t.timestamp,
        ROW_NUMBER() OVER (
            PARTITION BY t.card_id
            ORDER BY t.timestamp DESC
        ) AS rn
    FROM transactions t
),
last_3 AS (
    SELECT *
    FROM ranked
    WHERE rn <= 3
)
SELECT
    card_id,
    CASE
        WHEN SUM(declined) = 3 THEN 'inactive'
        ELSE 'active'
    END AS status
FROM last_3
GROUP BY card_id;
```

Output

Action Output

#	Time	Action	Message
72	12:58:00	CREATE TABLE card_status AS WITH ranked AS (SELECT t.card_id, t.declin...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0

```sql
SELECT COUNT(*) AS active_cards
FROM card_status
WHERE status = 'active';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

active_cards
4995

Result 7

Output

Action Output

#	Time	Action	Message
72	12:58:00	CREATE TABLE card_status AS WITH ranked AS (SELECT t.card_id, t.declin...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
73	12:58:13	SELECT COUNT(*) AS active_cards FROM card_status WHERE status = 'active'	1 row(s) returned

En el Código anterior se crea una nueva tabla llamada card_status, asi mismo hago uso de una instrucción de la función ventana ROW_NUMBER, donde se toma los datos de la tabla transactions para cada tarjeta. Asi asignar “rn” a cada transacción, posteriormente paso a filtrar las ultimas 3 transacciones de las tarjetas.

Nivel 3: Crea una tabla con la que podamos adjuntar los datos del nuevo archivo products.csv con la base de datos creada, dado que desde la transacción tienes product_ids. Genera la siguiente consulta: Ejercicio 1 Necesitamos saber el número de veces que se ha vendido cada producto.

```sql
CREATE TABLE transaction_products (
    transaction_id VARCHAR(50),
    product_id INT,
    PRIMARY KEY (transaction_id, product_id)
);
INSERT INTO transaction_products (transaction_id, product_id)
SELECT
    t.id,
    jt.product_id
FROM transactions t,
JSON_TABLE(
    CONCAT('[', t.product_ids, ']'),
    "$[*]" COLUMNS(product_id INT PATH "$")
) AS jt;
SELECT
```

Output

Action Output

#	Time	Action	Message
74	13:03:03	CREATE TABLE transaction_products (transaction_id VARCHAR(50), product_id INT, ...	0 row(s) affected
75	13:03:13	INSERT INTO transaction_products (transaction_id, product_id) SELECT t.id, jt.produc...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0

```sql
SELECT
    p.id,
    p.product_name,
    COUNT(tp.product_id) AS times_sold
FROM transaction_products tp
JOIN products p ON p.id = tp.product_id
GROUP BY p.id, p.product_name
ORDER BY times_sold DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	product_name	times_sold
5	skywalker ewok	2548
79	Direwolf riverland...	2545
84	Karstark Tully Dorne	2545
78	Karstark warden	2543
85	bastard of north	2539
57	Direwolf Stannis ri...	2539
89	skywalker ewok	2538
56	Direwolf riverlands	2537

Result 8

Output

Action Output

#	Time	Action	Message
75	13:03:13	INSERT INTO transaction_products (transaction_id, product_id) SELECT t.id, jt.produc...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0
76	13:04:05	SELECT p.id, p.product_name, COUNT(tp.product_id) AS times_sold FROM transacti...	100 row(s) returned

Es preciso crear una tabla intermedia por medio de json, por tanto, se crea una tabla llamada transaction_products como tabla relacional, pero como un paso intermedio, cada fila representa un producto dentro de una transacción. Este código transforma datos almacenados en formato JSON dentro de una tabla en una estructura relacional, para analizarlos mejor; entonces convierte listas de productos por transacción en filas individuales y calcula la frecuencia de venta de cada producto.